# Optimization Models for Targeted Offers in Direct Marketing

## Implementation and computational study of the exact formulations

**after F. Talla Nobibon, R. Leus, F.C.R. Spieksma, *EJOR* 210 (2011) 670–683**

Judecir Cavalcante
ist431224

PhD Programme in Engineering and Management
Department of Engineering and Management
Instituto Superior Técnico, Universidade de Lisboa

Optimization and Applications
Prof. José Rui Figueira
Homework 3

*Problem and models follow [1].* **Blue** *marks the presenter's implementation, experiments and interpretation.*

# Outline


1. Problem and data
2. Formulations
3. Implementation
4. Experimental design
5. Experiment 1: small instances
6. Experiment 2: groups S1 to S3
7. Summary

# The problem

- A **direct-marketing campaign** offers products to selected customers. The paper addresses *retention* campaigns, which seek additional revenue from existing customers of a bank [1].
- Response models estimate the expected return $p_{ij} = r_{ij}\, DFV_{ij}$ of offering product $j$ to customer $i$ (acceptance probability times value to the firm); these estimates are **inputs** of the model.

## Product targeting problem

Decide which products to offer to which customers so as to maximize the expected net profit of the campaign, subject to four business requirements:

1. a corporate **hurdle rate** $R$ on the return on investment;
2. a **budget** $B_j$ per product, because each business unit funds its own product;
3. a maximum number $M_i$ of **offers per customer**;
4. a **minimum-quantity commitment** (MQC) $O_j$: a product used in the campaign is offered to at least $O_j$ customers.

A **fixed cost** $f_j$ is incurred whenever product $j$ is used.

The problem is strongly NP-hard; even finding a solution with positive profit is NP-hard [1].

# Implementation: Instance generation

Instances follow the rules of Section 5.1 of [1], with $C_j = \sum_i c_{ij}$ and $\bar{M} = \sum_i M_i/n$.

| Parameter | Meaning | Rule |
|---|---|---|
| $c_{ij}, p_{ij}$ | cost and expected return of an offer | integers in $\{1, 2, 3\}$ and $\{0, \ldots, 16\}$ |
| $R$ | hurdle rate | $5\%$, $10\%$ or $15\%$ |
| $M_i$, variant s | few offers per customer | between $1$ and $n/5$ |
| $M_i$, variant $\ell$ | many offers per customer | between $\lceil n/3 \rceil$ and $\lceil 2n/3 \rceil$ |
| $O_j$ | minimum-quantity commitment | between $\bar{M}$ and $2\bar{M}$ |
| $B_j$ | budget: minimum, maximum or random | between $O_j C_j/m$ and $2\bar{M}C_j/m$ |
| $f_j$ | fixed cost | proportional to $O_j$ and to the margin of product $j$ |

## Implementation

- The generator follows these rules, with two small adjustments (rounding of $O_j$ and range of the random budget) that keep the budgets consistent.
- Instances are reproducible from a fixed seed. Because the seed is the same, instances with the same $n$ share their costs and returns.

# Model M1

Variables: $x_{ij} = 1$ if product $j$ is offered to customer $i$; $y_j = 1$ if product $j$ is used in the campaign.

$$\max \sum_{i,j} (p_{ij} - c_{ij})\, x_{ij} - \sum_j f_j\, y_j \qquad (1)\ \text{net profit}$$

$$\text{s.t.} \sum_{i,j} p_{ij} x_{ij} \;\geq\; (1+R)\Big[\sum_{i,j} c_{ij} x_{ij} + \sum_j f_j y_j\Big] \qquad (2)\ \text{hurdle rate}$$

$$\sum_i c_{ij} x_{ij} \leq B_j, \qquad j = 1, \ldots, n \quad (3)\ \text{budget}$$

$$\sum_j x_{ij} \leq M_i, \qquad i = 1, \ldots, m \quad (4)\ \text{offers per customer}$$

$$\sum_i x_{ij} \leq m\, y_j, \qquad j = 1, \ldots, n \quad (5)\ \text{linking}$$

$$\sum_i x_{ij} \geq O_j\, y_j, \qquad j = 1, \ldots, n \quad (6)\ \text{MQC}$$

$$x_{ij},\ y_j \in \{0, 1\}. \qquad (7)$$

- (M1) has $mn + n$ binary variables and $1 + m + 3n$ constraints. Constraints (5) and (6) together state that a product is either unused or offered to at least $O_j$ customers.
- In the LP relaxation, (5) $\sum_i x_{ij} \leq m\, y_j$ allows $y_j = \sum_i x_{ij}/m$, so the fixed cost $f_j$ is charged at a rate of only $f_j/m$ per offer. This weakens the LP bound of (M1).

# Model M2

(M2) replaces the aggregated linking constraints (5) $\sum_i x_{ij} \leq m\, y_j$ by their **disaggregated** form

$$x_{ij} \leq y_j, \qquad i = 1, \ldots, m,\ j = 1, \ldots, n. \qquad (8)$$

## Proposition 3 [1]

The convex hull of the feasible solutions of (M1) equals the convex hull of the solutions satisfying (2)–(4), (6) and (8), with $0 \leq y_j \leq 1$ and $x_{ij} \in \{0, 1\}$.

- Summing (8) over $i$ gives (5); hence every LP solution of (M2) is an LP solution of (M1), and $z^{LP}_{(M2)} \leq z^{LP}_{(M1)}$.
- In the paper, (M2) also relaxes $y_j \in \{0, 1\}$ to $0 \leq y_j \leq 1$. It has $mn - n$ more constraints than (M1): $mn$ constraints (8) replace the $n$ constraints (5).
- In the paper, (M2) is the formulation solved with CPLEX 10.2.

**Remark** *(implementation)*

The implementation keeps $y_j \in \{0, 1\}$ in the integer version of (M2). By Proposition 3 this does not change the optimal value, and the LP relaxation is the same, since $x$ and $y$ are relaxed to $[0, 1]$ in both cases.

# Model M3

For each product $j$, let $S_{pj}, p = 1, \ldots, k_j$, be the sets of customers with $|S_{pj}| \geq O_j$ and $\sum_{i \in S_{pj}} c_{ij} \leq B_j$. The binary variable $z_{pj}$ equals 1 if product $j$ is offered exactly to $S_{pj}$.

$$\max \sum_j \sum_p \left[ \sum_{i \in S_{pj}} (p_{ij} - c_{ij}) - f_j \right] z_{pj} \qquad (9)$$

$$\text{s.t.} \sum_j \sum_p \left[ \sum_{i \in S_{pj}} \big(p_{ij} - (1+R)c_{ij}\big) - (1+R) f_j \right] z_{pj} \geq 0 \qquad (10) \text{ hurdle rate}$$

$$\sum_j \sum_{p:\, i \in S_{pj}} z_{pj} \leq M_i, \qquad i = 1, \ldots, m \qquad (11) \text{ offers per customer}$$

$$\sum_{p=1}^{k_j} z_{pj} \leq 1, \qquad j = 1, \ldots, n \qquad (12) \text{ one set per product}$$

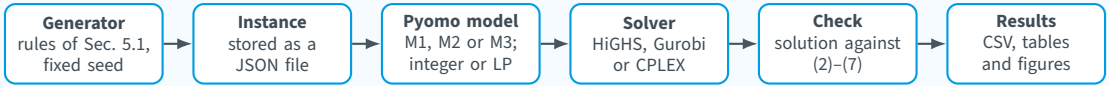
$$z_{pj} \in \{0, 1\}. \qquad (13)$$

- Budget (3) and MQC (6) are built into the definition of the columns. (M3) results from a Dantzig–Wolfe decomposition, and $z^\star \leq z^{LP}_{(M3)} \leq z^{LP}_{(M2)}$ [1].
- The number of columns grows as $O(n\, 2^m)$. The paper solves (M3) by column generation within branch-and-price.

**Remark** *(implementation)*

The implementation enumerates all admissible columns explicitly and is therefore restricted to $m \leq 12$. Branch-and-price is not implemented. The solution of (M3) is mapped back to $(x, y)$ and checked against (2)–(7).

# Implementation: Overview




- **Pyomo as a common modelling layer.** Each formulation is written once, in algebraic form close to the paper, and sent to HiGHS, Gurobi or CPLEX by changing only the solver name.
- **M1 and M2 share one model builder**; they differ only in the linking constraints, (5) or (8). The LP relaxation changes the domain of the variables from $\{0, 1\}$ to $[0, 1]$.
- **M3 in two steps:** all admissible columns are enumerated, then the set-covering model is built from them.
- **Every solution is checked** against (2)–(7) by an independent function, and unit tests cover a small example solved by brute force.

# Design: Experiments

**Experiment 1** *(small instances)*

- 50 instances with $m = 4, \ldots, 12$ customers and $n = 2, \ldots, 5$ products.
- Models **M1, M2 and M3**.
- Goal: validate the three models and compare their LP bounds.

**Experiment 2** *(groups S1, S2, S3 of the paper)*

- 162 instances with $m = 100$, 200, 300 customers and $n = 5$, 10, 15 products.
- Models **M1 and M2** (M3 needs column generation at this size).
- Goal: compare bounds and solve times as $m$ grows.

In both experiments, each instance is solved as an integer program and as an LP relaxation, with **HiGHS, Gurobi and CPLEX**.

**Computational environment**

| | |
|---|---|
| Machine | Intel Core i7-8565U @ 1.80 GHz (4 cores, 8 threads), 15.8 GB RAM, Windows 10 |
| Software | Python 3.12.10, Pyomo 6.10.1 |
| Solvers | HiGHS 1.15.1, Gurobi 13.0.3, CPLEX 22.2.0.0, with default settings |
| Time limit | 300 s (Experiment 1) and 3600 s (Experiment 2) per run |

# Experiment 1: LP bounds

- The three models and the three solvers return the same optimum $z^\star$ on all 50 instances.
- The LP gap $100\,(z^{LP} - z^\star)/z^\star$ depends only on the model, and $z^\star \le z^{LP}_{(M3)} \le z^{LP}_{(M2)} \le z^{LP}_{(M1)}$ holds in every instance.

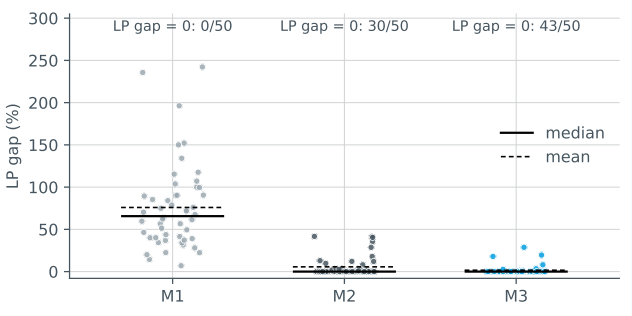



**Figure 1.** LP gap of M1, M2 and M3 on the 50 instances of Experiment 1; one point per instance.

| | LP gap (%) | | | LP gap |
|---|---|---|---|---|
| Model | mean | median | max | $= 0$ |
| M1 | 75.90 | 65.63 | 242.22 | 0/50 |
| M2 | 5.63 | 0.00 | 41.67 | 30/50 |
| M3 | 1.63 | 0.00 | 28.75 | 43/50 |

The LP relaxation of M3 already gives the integer optimum in 43 of 50 instances.

# Experiment 1: Size of M3

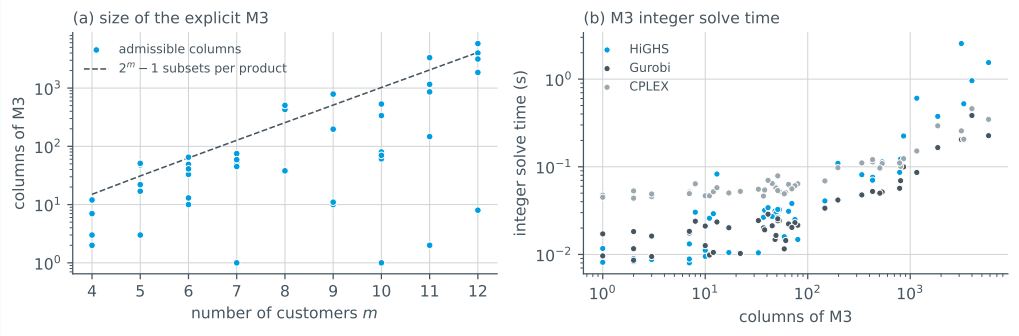



**Figure 2.** (a) Admissible columns of M3 per instance, as a function of $m$. (b) Integer solve time of M3 against its number of columns, per solver. Logarithmic scales.

- The number of columns ranges from 1 to 5 809, already with at most 12 customers.
- The solve time of M3 increases with the number of columns, but stays below 2.6 s.

# Experiment 1: Solve times

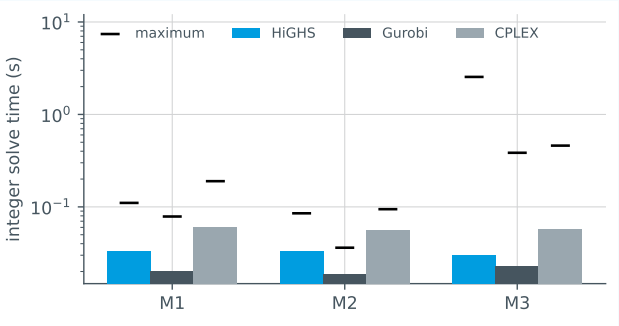



**Figure 3.** Integer solve time by solver and model in Experiment 1: bars show the median over 50 instances, marks the maximum. Logarithmic scale.

| | | time (s) | |
|---|---|---|---|
| Solver | Model | median | max |
| HiGHS | M1 | 0.033 | 0.110 |
| | M2 | 0.033 | 0.085 |
| | M3 | 0.030 | 2.546 |
| Gurobi | M1 | 0.020 | 0.079 |
| | M2 | 0.019 | 0.036 |
| | M3 | 0.023 | 0.385 |
| CPLEX | M1 | 0.060 | 0.190 |
| | M2 | 0.056 | 0.094 |
| | M3 | 0.058 | 0.460 |

- All instances are solved in a fraction of a second, except M3 on the instances with many columns.
- At this size, the times are too small to rank the solvers.

# Experiment 1: Conclusions

1. **The three models are correctly implemented**: they reach the same optimum on all 50 instances, with the three solvers.
2. **The theoretical ordering of the LP bounds is confirmed**: $z^\star \leq z^{LP}_{(M3)} \leq z^{LP}_{(M2)} \leq z^{LP}_{(M1)}$.
3. **Disaggregation is the main gain**: the mean LP gap falls from 75.90% (M1) to 5.63% (M2), and to 1.63% with M3.
4. **The strength of M3 has a size cost**: thousands of columns already for $m \leq 12$. Larger instances require column generation, as in the paper.

# Experiment 2: Optimality

- **All runs finish with status optimal**: 162 instances $\times$ 2 models $\times$ 3 solvers $= 972$ integer runs, and the same number of LP runs.
- **No run reaches the time limit** of 3600 s; the longest takes 1 145 s.
- **The models and the solvers agree on the optimum.** The only exceptions are three runs on S3 that stop at the default tolerance of the solver ($0.01\%$), one unit away from the optimum (for example 10 898 instead of 10 899).
- The optimum $z^\star$ of each instance is therefore taken as the best value over its six runs.

# Experiment 2: LP bounds

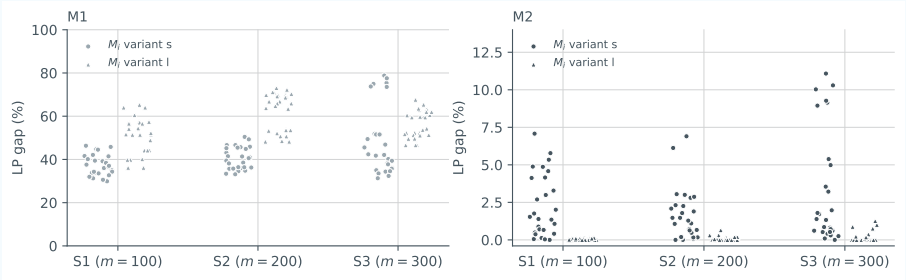



**Figure 4.** LP gap of M1 (left) and M2 (right) on the 162 instances, by group and $M_i$ variant. The vertical scales differ.

| Group | M1: LP gap (%) mean | M1: LP gap (%) max | M2: LP gap (%) mean | M2: LP gap (%) max |
|---|---|---|---|---|
| S1 ($m = 100$) | 44.36 | 65.30 | 1.18 | 7.08 |
| S2 ($m = 200$) | 51.61 | 73.09 | 0.87 | 6.90 |
| S3 ($m = 300$) | 52.24 | 78.82 | 1.76 | 11.08 |

- The LP gap of M2 is about 1% on average, against 44–52% for M1, in every group.

# Experiment 2: Solve times

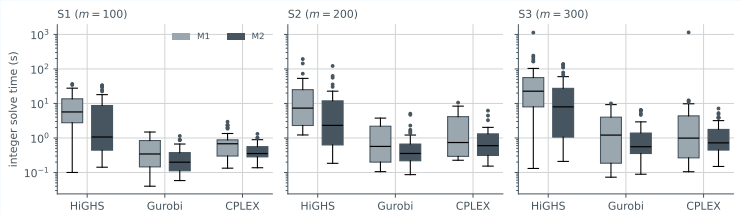



**Figure 5.** Integer solve time of M1 and M2 per solver, for each group (54 instances per box). Logarithmic scale.

| Solver | Model | S1 ($m = 100$) median | S1 ($m = 100$) max | S2 ($m = 200$) median | S2 ($m = 200$) max | S3 ($m = 300$) median | S3 ($m = 300$) max |
|---|---|---|---|---|---|---|---|
| HiGHS | M1 | 5.68 | 36.71 | 7.33 | 193.25 | 22.65 | 1 128.02 |
| | M2 | 1.07 | 33.85 | 2.32 | 122.02 | 7.97 | 139.14 |
| Gurobi | M1 | 0.34 | 1.49 | 0.57 | 3.76 | 1.21 | 10.02 |
| | M2 | 0.20 | 1.14 | 0.36 | 5.06 | 0.56 | 6.50 |
| CPLEX | M1 | 0.68 | 2.95 | 0.74 | 10.59 | 0.99 | 1 145.38 |
| | M2 | 0.35 | 1.31 | 0.60 | 6.21 | 0.72 | 7.15 |

- Gurobi and CPLEX solve a typical instance in about one second; HiGHS is roughly 7 to 9 times slower than Gurobi on M2.

# Experiment 2: Effect of the number of customers

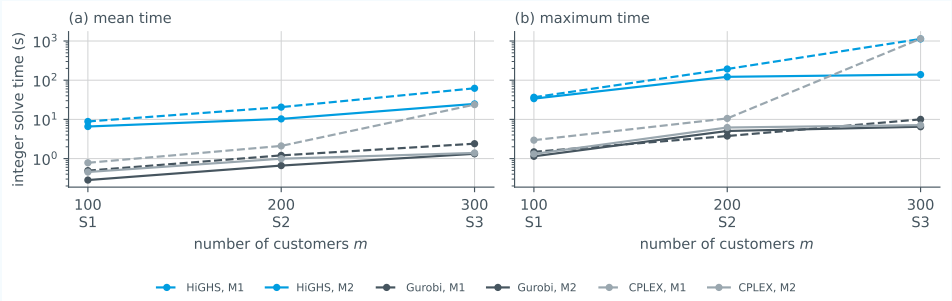



**Figure 6.** Mean (a) and maximum (b) integer solve time over the 54 instances of each group, per solver and model. Logarithmic scale.

- Mean times grow moderately with $m$: for M2, from 0.28 to 1.32 s with Gurobi and from 6.58 to 24.82 s with HiGHS.
- The worst case grows much faster for M1: its longest runs on S3 take about 1 100 s (CPLEX and HiGHS), while no M2 run exceeds 139 s.
- Each run was executed once, so the extreme values should be read with caution.

# Experiment 2: M1 versus M2

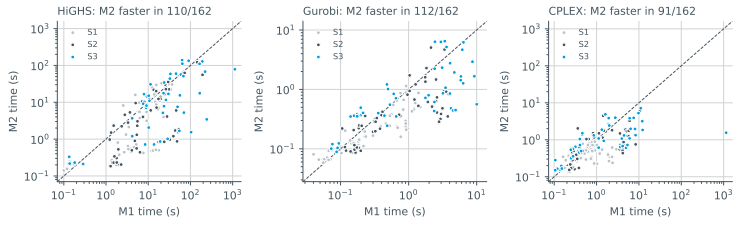



**Figure 7.** Integer solve time of M2 against M1 for each instance, per solver and group (logarithmic scales). Points below the diagonal are instances solved faster with M2.

| | instances where M2 is faster | | | geometric mean of time(M2)/time(M1) | | |
|---|---|---|---|---|---|---|
| Group | HiGHS | Gurobi | CPLEX | HiGHS | Gurobi | CPLEX |
| S1 | 35/54 | 44/54 | 36/54 | 0.46 | 0.66 | 0.69 |
| S2 | 43/54 | 41/54 | 34/54 | 0.31 | 0.59 | 0.65 |
| S3 | 32/54 | 27/54 | 21/54 | 0.44 | 0.71 | 0.75 |

- M2 is faster on average with every solver, but not on every instance.
- The time difference is much smaller than the difference in LP gap: the solvers probably strengthen M1 on their own (CPLEX solves 155 of the 162 M1 instances at the root node).

# Experiment 2: Small versus large $M_i$

Variant s: each customer accepts few offers. Variant $\ell$: each customer accepts many. Each variant has 81 instances.

| | mean LP gap (%) | | mean time of M2 (s) | | |
|---|---|---|---|---|---|
| Variant | M1 | M2 | HiGHS | Gurobi | CPLEX |
| s (few offers) | 42.05 | 2.43 | 24.74 | 1.14 | 1.35 |
| $\ell$ (many offers) | 56.76 | 0.10 | 3.05 | 0.37 | 0.55 |

- **Variant s is the harder case**: larger LP gap of M2 and solve times 2.5 to 8 times longer.
- **In variant $\ell$, the LP relaxation of M2 is almost integral** (zero gap in 45 of 81 instances), and M2 gains most over M1.
- A possible explanation: when customers accept few offers, the products compete for the same customers.

# Experiment 2: Conclusions

1. **Groups S1 to S3 are solved to optimality on a laptop**, with both models and the three solvers.
2. **M2 gives a much better bound than M1**: mean LP gap of about 1% against 44–52%, as in Experiment 1.
3. **M2 is faster on average and more robust**: its worst case is 139 s, against about 1 100 s for M1.
4. **The solver matters**: Gurobi and CPLEX take about one second per instance, HiGHS several times longer.
5. **Instances where customers accept few offers are the harder ones.**

# Summary: Conclusions and limitations

**Conclusions**

- The three exact formulations of the paper were implemented and validated: all models and solvers agree on the optimum.
- The LP bounds follow the theory: M3 is the strongest and M1 by far the weakest. The disaggregated constraints (8) give most of the improvement.
- Up to $m = 300$, the compact model M2 with a commercial solver is enough: at most 7.2 s per instance.

**Limitations**

- M3 is solved only by enumeration ($m \leq 12$); column generation and branch-and-price are not implemented.
- Only the small groups (S1 to S3) were run, with one seed and one run per instance.
- The instances are generated by the rules of the paper, but they are not the paper's instances.

# Thank you

Questions?

**Judecir Cavalcante** (ist431224)
PhD Programme in Engineering and Management
Department of Engineering and Management
Instituto Superior Técnico, Universidade de Lisboa

Optimization and Applications
Prof. José Rui Figueira
Homework 3

# Reference

[1] F. Talla Nobibon, R. Leus, F.C.R. Spieksma. Optimization models for targeted offers in direct marketing: Exact and heuristic algorithms. *European Journal of Operational Research*, 210(3):670–683, 2011.